Do Generate–Verify–Repair Guards Improve LLM NetOps Outputs? A Paired Emulator Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory experiment (n=200 natural-language NetOps intents; study_id cnd-001-5f3a) that compares ungated LLM generation to a bounded generate–verify–repair (GVR) pipeline applied to a single shared LLM candidate per intent. Generation used gpt-5-mini and the guarded arm applied a deterministic three-stage validator and a deterministic, rule-based repair operator with repair_budget <=1 (no extra LLM calls). Artifact-backed primary outcomes: baseline successes = 199/200 (0.995), guarded = 200/200 (1.000); paired contingency n11=199, n10=1, n01=0, n00=0; paired difference (guarded - baseline) = 0.005. The preregistered exact two-sided McNemar (exact binomial on discordant pairs) yields p = 1.0; paired-bootstrap (10,000 resamples, seed 1729) 95% percentile CI = [0.0, 0.015]. The preregistered 0.15 absolute-effect target is excluded by the observed CI because the realized baseline was near-ceiling. We provide concise, auditable descriptions of the validator and repair operator, execution accounting (model_calls_used = 201; deterministic repair invocations = 1), exact-test justification appropriate for small discordant counts, and operationally grounded limitations. All prompts, provider traces, validator traces, repair traces (the single invoked repair trace), and analysis artifacts are archived in the study evidence bundle.**

I. Introduction

Automating routine network-operations (NetOps) changes with large language models (LLMs) promises reduced manual effort but raises an operational-safety question: do LLM-produced configuration artifacts execute correctly when applied to control planes? Prior work demonstrates intent-to-configuration prototypes and documents structured-output failures (missing commands, misordering, protected-field modification) and proposes mitigations such as retrieval-augmentation, constrained decoding, verifier-assisted repair, and multi-stage tool-augmentation. Despite these proposals, there is limited execution-grounded evidence that quantifies the marginal benefit of applying a bounded deterministic repair to the same initial LLM candidate (a low-hosted-call operational estimand).

We preregistered and executed a paired experiment (study_id cnd-001-5f3a) that isolates the "repair-on-same-candidate" question by sharing a single initial LLM candidate across both arms (baseline ungated acceptance and guarded generate–verify–repair). The shared-candidate estimand is operationally relevant when hosted-model call budgets are constrained. Our contributions are: (1) a preregistered, artifact-backed paired evaluation under the shared-candidate estimand; (2) concise algorithmic descriptions of the deterministic three-stage validator and the deterministic, rule-based repair operator used in the guarded pipeline; and (3) complete execution accounting and exact-test justification for small-discordant-pair inference.

II. Related Work

LLMs have been extensively explored for NetOps tasks including intent translation, co-pilot assistance, and configuration synthesis; surveys and prototype studies document both the promise and the recurring problem of structured-output failures. Cross-domain mitigation methods (RAG, live schema grounding, constrained decoding) reduce hallucinations in NL2SQL and document-grounded tasks but require adaptation to device-specific CLIs. Formal-verification formalisms (NetKAT variants, weighted NetKAT) and ensemble verifiers provide deterministic checks for reachability and isolation, making them natural validators for generated configurations, although scalability and fidelity to control-plane dynamics remain open issues [doi:10.1145/3808318]. Architectural treatments of agentic NetOps emphasize assurance machinery—auditable evidence traces, bounded tool use, rollback policies—as essential to operational reliability. Our study addresses a narrow empirical gap (G2 in our evidence synthesis): an execution-grounded measurement of how much deterministic repair-on-the-same-candidate improves emulator-executable success under a shared-generation budget.

III. Methodology

Preregistration and design: The confirmatory protocol (study_id cnd-001-5f3a) was preregistered with a paired, shared-initial-generation design: n=200 curated natural-language NetOps intents (single-device and small multi-device changes such as VLAN/MTU updates and uplink switchover). For each intent the hosted model produced one initial candidate used by both arms. Baseline accepts that candidate; guarded runs a deterministic validator and applies at most one deterministic repair when the validator reports a repairable violation. The primary estimand is the paired_success_rate_difference_guarded_minus_baseline; the preregistered primary test is McNemar's paired binary test implemented in its exact-binomial (exact McNemar) form appropriate for small discordant counts. Planned maximum hosted-model calls = 400; realized model_calls_used = 201 (shared_initial_generation = 200; repair-stage calls = 1). Execution used Dockerized Mininet + FRR and deterministic_netops_validator_v5 as the validator.

Task bank and scoring: The confirmatory bank consists of 200 curated intent–expected-configuration pairs with

explicit workflow constraints and protected-interface rules. The full_intent_constraint_validation scoring rule requires: (a) syntactic parse; (b) presence of the task's required_sequence; (c) adherence to dependency and group policies (make-before-break, shutdown_configure_restore, minimum-active constraints); (d) preservation of protected interfaces; and (e) emulator-observed final_state equality with expected_final_state. Provider/API generation failures were predeclared as failures for an arm; none occurred in confirmatory runs.

Deterministic validator (three-stage description): The validator deterministically executes three ordered stages and emits a machine-readable trace used for scoring and repair decisions. Stage 1—syntactic parse and canonical normalization—tokenizes CLI-like lines into normalized_configuration and flags SYNTAX_PARSE_ERROR on failure. Stage 2—structural/workflow checks—verifies required commands, ordering/dependency constraints, group-activity policies, and protected-interface rules; violations are emitted as deterministic codes (e.g., MISSING_REQUIRED_COMMAND, DEPENDENCY_VIOLATION, PROTECTED_INTERFACE_MODIFICATION). Stage 3—deterministic emulator application—applies the normalized_configuration to the local emulator to produce observed final_state and compares it to expected_final_state. Validator traces contain parsed_command_count, observed_touched_field_count, violation_count, normalized_configuration, and validation_after.valid; all traces are archived for audit and analysis.

Deterministic repair operator (concise algorithmic description): The repair operator is a deterministic, rule-based function invoked only when the validator reports violation_count > 0 and the violation family is within a preregistered repairable set. Operational constraints enforced by the preregistration: (1) no additional LLM calls—repairs are implemented by deterministic code; (2) bounded budget—at most one repair per episode (repair_budget <= 1) for the confirmatory run; (3) unambiguous mapping—repairs apply only when the mapping from violation code to deterministic transformation is unique. The operator implements three canonical deterministic transformations: - Insertion: insert missing required commands derived from the task's required_sequence (e.g., insert shutdown/restore commands to satisfy shutdown_configure_restore), placed deterministically at canonical positions based on the task payload. - Reordering: atomically reorder annotated command blocks to satisfy dependency constraints while preserving command group boundaries. - Preservation enforcement: redact or replace commands that would violate preserve_unspecified_state or protected-interface rules with the original values deterministically. Ambiguous violations (multiple plausible insert positions) or nonrepairable violation families are not modified and remain failures. All repair decisions and the resulting normalized_configuration are recorded in a repair trace archived with the run artifacts.

Statistical analysis: The primary test is McNemar's test for paired binary outcomes implemented via the exact binomial formulation: with n = n10 + n01 the number of discordant pairs, the test computes the two-sided binomial tail probability under Binomial(n, 0.5). This exact formulation is appropriate when discordant counts are small (here n = 1). Complementary uncertainty quantification uses paired-bootstrap percentile 95% CIs with 10,000 resamples (bootstrap_seed = 1729) for the paired difference. The preregistered power calculation targeted an absolute effect of 0.15 at n=200 assuming baseline $\approx$0.5; realized near-ceiling baseline substantially reduces detectable discordant counts and thus inferential sensitivity.

IV. Results

Execution and primary outcomes: The confirmatory run completed all planned episodes (400 episodes = 200 paired intents) with no provider/API failures. Condition-level artifact-backed counts (analysis/condition_summary.csv) show baseline_successes = 199/200 (0.995) and guarded_successes = 200/200 (1.000). The paired contingency (analysis/paired_contingency_table.csv) is n11 = 199, n10 = 1, n01 = 0, n00 = 0; paired difference (guarded - baseline) = 0.005.

Exact-test and CI details: The preregistered exact two-sided McNemar (exact binomial on discordant pairs) was computed on the observed discordant count n = n10 + n01 = 1; the resulting two-sided p-value reported by the analysis executor is p = 1.0 (analysis/results.json). Complementary uncertainty quantification used a paired-bootstrap percentile CI with 10,000 resamples (bootstrap_seed = 1729) yielding a 95% CI = [0.0, 0.015]. Because the preregistered primary effect-size target (absolute 0.15) lies well outside the observed CI, the confirmatory data do not support that magnitude of improvement under the shared-candidate estimand.

Repair and failure-accounting diagnostics: The validator reported validation_before.valid == false in exactly one confirmatory episode; deterministic repair invocations = 1 and the single deterministic repair converted that episode into a validator-passing final configuration, producing the sole guarded-only success (n10 = 1). Across the confirmatory bank syntactic/parse failures = 0 and no provider-call timeouts or API errors were recorded. Provider-call accounting (deterministic_reconciliation.json and execution/model_configuration.json) records hosted_model_call_event_count = 201, completed_hosted_model_call_event_count = 201, cache_miss_count = 201, stage_counts = {shared_initial_generation: 200, repair: 1}.

Representative exemplars and difficulty context: To illustrate task difficulty and typical intents we publish six representative exemplars with the validator traces and difficulty annotations (the execution/task_manifest.jsonl archive contains the full 200-entry manifest). The six exemplars included in the evidence bundle show difficulty labels: 1 medium (shutdown_configure_restore pattern), 2 hard (make-before-break uplink switchover; paired atomic MTU change), and

3 very_hard (safe-access/service-transfer and rolling migration patterns). These exemplars expose the kinds of dependency and group-activity constraints present in the confirmatory set; the full per-episode task manifest (execution/task_manifest.jsonl) contains complete per-task difficulty metadata for re-analysis.

Artifact-grounded diagnostics: All per-episode scoring JSONL artifacts include validator_trace and scoring_reason fields (e.g., execution/scoring/task-00000X-*.json). The paired_contingency_table.csv and condition_summary.csv (analysis/) provide the exact counts used by the primary analysis. The analysis_log.jsonl records bootstrap parameters (resamples = 10,000, seed = 1729) and the exact McNemar computation event. The deterministic_reconciliation.json documents end-to-end episode accounting and verifies that completed_episode_count = 400, complete_pair_count = 200, and that no episodes were excluded for contamination.

Interpretation of the small-discordant outcome: With only one discordant pair (n = 1), the exact McNemar test conveys that the observed marginal improvement (one converted episode) is not unlikely under the null in two-sided terms given the small n; the paired-bootstrap CI conveys the scale of uncertainty around the paired difference (upper bound 0.015). Practically, these diagnostics show that deterministic repair-on-the-same-candidate delivered an auditable, low-cost correction in exactly one case in this curated confirmatory bank; in higher-error regimes or under independent-resampling estimands the repair yield and statistical sensitivity could differ substantially.

Sensitivity notes and archived artifacts for reanalysis: The preregistered missingness plan treated provider/API failures as failures for the arm; because none occurred the primary analysis is not affected by missingness imputation. The preregistered power plan targeted a 0.15 absolute improvement assuming baseline$\approx$0.5; the realized near-ceiling baseline compresses discordant counts and reduces power to detect large absolute effects under this estimand. All relevant artifacts for re-analysis are archived (execution/*, analysis/*, and the full task manifest), enabling investigators to recompute alternative estimands (e.g., independent-resampling) or to inspect the single repaired episode and validator traces for mechanistic insight.

V. Discussion

Operational interpretation: Under the shared-initial-generation estimand and the curated confirmatory distribution the ungated gpt-5-mini generator produced near-perfect candidates (199/200). Deterministic, auditable validators produce structured violation codes that enable targeted deterministic repairs; in low-error regimes the marginal yield of a conservative rule-based repair on the same candidate is small but auditable and low-cost. For operators constrained by model-call budgets, the guarded shared-candidate design preserves low hosted-call cost while providing an assurance gate; for contexts prioritizing maximal success, independent resampling or LLM-invoked repair loops merit exploration.

Design-space trade-offs and practitioner guidance: The confirmatory run's execution accounting (model_calls_used = 201; repair invocations = 1; completed_episodes = 400) enables simple cost-versus-yield comparisons. If operators accept higher hosted-call expense, independent-resampling (two or more independent LLM generations per intent) or LLM-guided iterative repair could raise success rates at the cost of greater provider usage and more complex governance. Deterministic validators should be considered mandatory assurance machinery where auditable, repeatable checks and low-latency failure explanations are required.

Threats to validity: Internal validity is strong for the paired estimand and deterministic scoring rules; construct validity depends on emulator fidelity—our deterministic emulator models control-plane state deterministically but cannot fully capture runtime effects such as BGP convergence timing or specific device firmware idiosyncrasies. External validity is limited by the single LLM (gpt-5-mini) and a synthetic curated bank focused on small changes; extrapolation to multivendor CLI diversity or adversarial/out-of-distribution intents is limited. Conclusion validity is constrained by the near-ceiling baseline that reduced discordant-pair counts; the exact McNemar is the correct small-sample test for this observed pattern but offers limited sensitivity when n is small. These limits are documented in the preregistration and archived manifests.

Reproducibility and auditability: All artifacts needed to reproduce analysis and inspection are archived in the evidence bundle: per-episode prompts and responses, provider-call traces, validator traces, deterministic repair trace (the single invoked repair), task manifest entries, analysis scripts, paired contingency CSV, bootstrap parameters (seed = 1729; resamples = 10,000), and execution manifest (preregistration_sha256: e5c5fdbd717c...). The exact-test implementation is the exact-binomial McNemar formulation applied to discordant-pair counts (n10,n01) and is described above; the analysis log records the p-value and bootstrap CI used in the manuscript.

VI. Limitations

- Synthetic, curated confirmatory task bank focused on small single- and multi-device changes; not comprehensive of production NetOps workloads (multivendor CLI dialects, hardware idiosyncrasies, dynamic control-plane timing).
- Single LLM (gpt-5-mini) evaluated; findings do not imply multi-model generality.
- Deterministic emulator + validator model control-plane behavior but cannot fully capture real-world runtime dynamics such as BGP convergence timing or vendor-specific runtime effects.
- Preregistered repair budget limited to a single deterministic repair iteration; different repair strategies or additional iterations might yield different outcomes.
- Shared-initial-generation design reduces model-call cost and isolates repair-on-same-candidate effectiveness

but reduces external generalizability compared with independent-resampling estimands.
- Near-ceiling baseline success limited statistical power to analyze failure-mode distributions in the confirmatory set; exploratory failure-taxonomy conclusions are therefore constrained.
- Adversarial or out-of-distribution intents (e.g., jailbreaks, malformed ambiguous intents) were not included in the confirmatory set and require separate evaluation.
- Full per-task difficulty label counts for all 200 confirmatory intents are recorded in execution/task_manifest.jsonl in the archived evidence bundle; the manuscript references that manifest rather than enumerating the full 200-line distribution inline to preserve concise presentation while preserving auditable reproducibility.

VII. Conclusion

We executed an autonomy-locked, preregistered paired experiment to measure whether a bounded generate–verify–repair guard improves emulator-executable success for LLM-generated NetOps configurations under a shared-candidate generation budget. Ungated gpt-5-mini generation succeeded in 199/200 confirmatory cases; the deterministic guarded pipeline succeeded in 200/200. The observed paired difference = 0.005 (exact McNemar two-sided p = 1.0; paired-bootstrap 95% CI [0.0, 0.015]) does not support the preregistered 0.15 absolute-effect target because the baseline was near-ceiling. For this estimand and task distribution, deterministic repair-on-same-candidate yielded negligible marginal improvement, but deterministic validators remain valuable, auditable assurance gates for operational NetOps pipelines. Full artifacts and analysis code are archived with the study for independent audit and re-analysis.

Disclosure Statement

This study was executed autonomously after an autonomy lock defined by the initial master prompt archived at provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d9 6940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human involvement prior to the lock was limited to selecting the topic family (Generative AI and LLMs for NetOps) and approving the master prompt. No manual human editing of technical content, figures, captions, references, results, or manuscript text occurred after the lock. The autonomous pipeline performed literature search and verification, research-gap selection, preregistration (study_id cnd-001-5f3a), code generation, experiment execution, artifact capture, analysis, internal critique, peer-review checks, and manuscript drafting.

Models and tooling: generation requested gpt-5-mini (declared_model_version in execution/model_configuration.json). Validation used deterministic_netops_validator_v5 implementing syntactic parsing, structural/workflow checks, and deterministic emulator application. Repairs in the confirmatory run were applied by deterministic code only (no additional LLM calls). The execution environment used Dockerized Mininet+FRR images orchestrated by adapter family hosted_netops_gvr_v1. Preregistered and enforced settings (not altered post-lock) include: primary estimand = paired_success_rate_difference_guarded_minus_baseline; confirmatory sample = n=200 paired intents; maximum model-call budget = 400; repair budget <=1 deterministic repair iteration; primary analysis = exact McNemar two-sided (exact-binomial on discordant pairs); bootstrap CIs = 10,000 resamples, seed = 1729. Execution accounting (artifact-backed): the run completed 400 episodes with model_calls_used = 201 (shared_initial_generation = 200; repair-stage calls = 1). All prompts, provider-call traces, responses, validator traces, repair traces (the single invoked repair trace), analysis artifacts, and the execution manifest are archived in the study evidence bundle.

References

[1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313

[2] Emmanuel Suárez Acevedo, Tiago Ferreira, Kevin Batz, Oliver Bøving, Nate Foster, Alexandra Silva, "Weighted NetKAT: A Programming Language for Quantitative Network Verification", 2026, 10.1145/3808318

[3] Sanjay Mishra, Divya Chukkapalli, Ganesh R. Naik, "Schema-Aware Localisation (SAL): Live Schema Grounding and Hallucination Validation for Oracle NL2SQL", 2026, 10.2139/ssrn.6909831

[4] Muhammad Bilal, Jon Crowcroft, Ruizhi Wang, Xiaolong Xu, Schahram Dustdar, "Large Language Models for Agentic NetOps and AIOps: Architectures, Evaluation, and Safety", 2026, 10.48550/arxiv.2605.12729

[5] Nilanjana Das, Edward Raff, Aman Chadha, Manas Gaur, "Human-Readable Adversarial Prompts: An Investigation into LLM Vulnerabilities Using Situational Context", 2026, 10.1145/3815174